

UNIVERSIDAD NACIONAL AUTONOMA DE MEXICO
FACULTAD DE ESTUDIOS SUPERIORES ARAGÓN

Asunto: Informe Global de Servicio Social

LIC. GUADALUPE MENDIETA BELLO
JEFA DEL DEPARTAMENTO DE SERVICIO SOCIAL

Mediante la presente, me dispongo a la enumeración de actividades llevadas a cabo durante el periodo de servicio social comprendido del 24 de febrero de 2025 al 11 de septiembre de 2025, dentro del programa "APOYO A PROYECTOS DE INVESTIGACIÓN DE LA FES ARAGÓN" con clave SS-2025-12/146-6671. Este documento tiene como propósito presentar de manera formal las labores realizadas y los avances obtenidos durante la participación en dicho programa.

Cabe recalcar que durante el tiempo de duración de este servicio social y la realización de este proyecto colabore con un compañero perteneciente a la carrera de Ingeniería en Computación por parte de la FES Aragón, Cristian Jair Cruz Sanchez. Con las bases de la colaboración planteadas y con la idea de crear un sistema que pudiera ayudar a erradicar la problemática vista en la institución acerca del manejo inadecuado de residuos se me fue dada la misión de realizar las siguientes tareas:

- Planeación

En un principio, nuestro equipo colaborativo en conjunto con la ayuda de nuestro responsable de proyecto, el M. en C. Arturo Ocampo Álvarez buscó el planteamiento de un proyecto que pudiera servir como apoyo a la comunidad estudiantil y al mismo tiempo brindarnos experiencia dentro del campo de la computación en el aspecto de Inteligencia Artificial. Después de este lapso de análisis acerca de las problemáticas escolares notamos cómo una en especial sobresaltaba en nuestros análisis, por lo tanto, la idea de realizar un sistema que pudiera ayudar a solventar un problema tan importante como la recolección y trato correcto de los residuos dentro de la institución fue planteada un punto a discutir con nuestro responsable de proyecto y con la aprobación de este comenzamos a trabajar en ella.

Durante este proceso de planeación puedo recalcar que el aprendizaje más destacable fue el aprender como funciona la planeación dentro de un ambiente compartido con personas de equiparable experiencia en este ambiente de trabajo y perteneciente a la carrera de Ingeniería en Computación, aparte de trabajar con la figura de un "jefe" como lo pudo ser nuestro responsable al cual debemos plantearle la propuesta, discutir tecnologías a usar, tiempos, actividades y diferentes puntos de la planeación total de proyecto.

- Investigación sobre técnicas de visión por computadora e inteligencia artificial aplicadas a la detección de basura.

Con la planeación completada y con los objetivos del proyecto planteados comencé una etapa de investigación, ya que después de plantear algunas rutas de trabajo con nuestro responsable, aun debimos escoger cuáles serían nuestras principales herramientas a utilizar a lo largo del proyecto y así plantear una ruta definida. YOLO (You Only Look Once) fue el modelo elegido por ambos miembros del equipo después de una extensa investigación, llegamos a la conclusión de que este modelo sería la mejor opción por su ya formada fama como el mejor modelo en tanto a técnicas de visión por computadora aparte de ser el mejor optimizado y con mayor compatibilidad.



En esta fase también a la par investigue y realice pruebas en plataformas como COLAB de Google que nos permitieran trabajar con la inmensa cantidad de recursos que se esperaban necesitar con la intención de elegir cual sería nuestro principal motor para el entrenamiento.

Esta fase contribuye bastante a mejorar mis habilidades de búsqueda y prueba en tanto a tecnologías de este nivel y a la importancia de elegir certeramente, con antelación y con conocimiento de las capacidades cada una de las tecnologías, aparte de encontrar nuevas plataformas de información, canales de difusión recomendados y tecnicas de planeacion para un proyecto.

- Recolección y preparación de datos para el dataset de imágenes necesario para el entrenamiento del modelo.

Esta fase del proyecto fue la más cargada de trabajo, al contar con todos los parámetros planteados, me dispuse a comenzar la recolección de datos con el cual se formará el dataset que sería la base para el entrenamiento del modelo.

Primero me enfoque en el correcto funcionamiento de un dataset y su formato para un modelo de detección en YOLOv8, para esto realice otra pequeña investigación buscando ejemplos de este y aprendiendo a como crear un correctamente y así finalmente pase a la fase más laboriosa.

Con todo listo dedique el tiempo completamente en recolectar imágenes que servirán como datos, ya fueran tomadas de internet o por mano propia mediante un dispositivo móvil, para este punto fueron de inmensa importancia paginas como Hugging Face, kaggle, roboFlow y extensiones como Fatkun Batch, las cuales me permitieron recolectar muchas imágenes y ejemplos de manera mucho más rápido más aparte de poder trabajar con las imágenes como al poder dividir entre formatos .jpg y .webp.

The screenshot shows the Kaggle Datasets page with a search for 'garbage'. The left sidebar contains navigation links: Home, Competitions, Datasets (selected), Models, Benchmarks, Code, Discussions, Learn, and More. The main content area displays a list of datasets related to 'garbage'. The search bar at the top shows 'garbage' and a 'Filters' button. Below the search bar, there are tabs for 'All datasets' and various categories like 'Computer Science', 'Education', 'Classification', 'Computer Vision', 'NLP', 'Data Visualization', and 'Pre-Trained Model'. The results show 370 datasets. The top results are:

Dataset Name	Updated	Usability	Files	Size	Downloads	Rank
Garbage Classification	sohanga - Updated 7 years ago	8.1	2532 Files (other)	86 MB	57.2K downloads	584
Garbage Dataset	Suman Kumar - Updated 9 months ago	10.0	10762 Files (other)	780 MB	13.5K downloads	83
Garbage Classification (12 classes)	Mostafa Mohamed - Updated 5 years ago	8.1	15515 Files (other)	251 MB	28.5K downloads	208
Garbage Image Dataset	Farzed Nekouei - Updated 2 years ago	5.6	2527 Files (other)	43 MB	7,810 downloads	103
Plastic - Paper - Garbage Bag Synthetic Images	Bolin - Updated 3 years ago	8.8	30000 Files (other, JSON)	473 MB	4,967 downloads	127
Split Garbage Dataset	Andrea Santoro - Updated 6 years ago					25

La siguiente fase consistió en trabajar los datos para conseguir el formato deseado, todo esto pude reducirlo mucho creando un script mediante código python que me ayudó a renombrar, acomodar y dividir la cantidad de más 22,000 imágenes de forma sencilla.

```
import os
from PIL import Image
import shutil

input_folder = r"C:\Users\alejandro\Downloads\xd"
output_folder = r"C:\Users\alejandro\Downloads\xd\salida"

os.makedirs(output_folder, exist_ok=True)

valid_exts = ('.jpg', '.jpeg', '.png')

prefix = "plastico_00007"
count = 0

for filename in sorted(os.listdir(input_folder)):
    if filename.lower().endswith(valid_exts):
        old_path = os.path.join(input_folder, filename)

        new_name = f"{prefix}{count:02d}.jpg"
        new_path = os.path.join(input_folder, new_name)

        if not filename.lower().endswith(".jpg"):
            img = Image.open(old_path).convert("RGB")
            img.save(new_path, "JPEG")
            os.remove(old_path)
        else:
            os.rename(old_path, new_path)

        label_name = new_name.replace(".jpg", ".txt")
        label_path = os.path.join(output_folder, label_name)

        with open(label_path, "w") as f:
            f.write("3 0.5 0.5 0.5 0.5\n")

        count += 1

print(f"Renombradas {count} imágenes y generadas sus etiquetas en '{output_folder}'")
```

Esta fase fue increíblemente laboriosa, pude comprender la importancia de conocer las tecnologías necesarias para automatizar procesos y acortar tiempos y gastos, me enseñó como esto puede ser crucial en la creación de cualquier tipo de proyecto y más en uno de este tipo y lo importante que será seguir actualizando mis conocimientos en tanto al uso de nuevas herramientas.

- Entrenamiento del modelo, evaluación de métricas, pruebas y ajustes para mejorar la precisión del modelo.

La parte más importante del proyecto fue esta, ya que nuestra base, el modelo sería creado en esta. Con el dataset completado me dispuse a realizar la primera prueba de entrenamiento la cual contaría con 5 épocas.

Esta prueba inicial me dio las primeras ideas de lo que sería el proceso completo de entrenamiento y con él la cual realice los primeros análisis de los datos.

```
from ultralytics import YOLO
import os
import shutil

model = YOLO('yolov8s.pt')

model.train(
    data= r'C:\Users\alejandro\Downloads\dataset_renamed_yolo\data.yaml',
    epochs=5,
    imgsz=640,
    batch=32
)
```

```
PS C:\Users\alejandro> & C:\Users\alejandro\AppData\Local\Programs\Python\Python313\python.exe c:/Users/alejandro/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 27.5±19.7 MB/s, size: 13.9 KB)
val: Scanning C:\Users\alejandro\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	4661	4661	0.869	0.859	0.941	0.929
organico	478	478	0.8	0.916	0.946	0.941
reciclable	2105	2105	0.935	0.844	0.959	0.946
otros	1340	1340	0.83	0.89	0.941	0.93
plastico	738	738	0.911	0.787	0.917	0.899

```

Speed: 0.7ms preprocess, 91.1ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val
Precisión media (mp): 0.8689179026599186
Recall medio (mr): 0.8591690833845309
mAP50: 0.9405667882189012
mAP50-95: 0.9290407483464488
Precisión por clase: [ 0.8 0.935 0.83009 0.91059]
Recall por clase: [ 0.91632 0.84418 0.88958 0.7866]
F1 por clase: [ 0.85422 0.88727 0.85881 0.84406]
mAP50 por clase: [ 0.94599 0.95892 0.94079 0.91656]
mAP50-95 por clase: [ 0.9415 0.94562 0.92993 0.89912]

```

Tras la finalización de las 5 primeras épocas y su análisis proseguí con un siguiente entrenamiento y sentamos la base de entrenar el modelo a partir de batches de 5 épocas, con esto el siguiente entrenamiento el cual completó 10 épocas me arrojó resultados que al analizarlos y comentarlos con mi compañero me hizo notar inconsistencias en el crecimiento de partes de nuestro modelo, en concreto una de las clases no parecía mejorar a la par, esto me llevó a sumarme a la tarea de a la par de realizar los entrenamientos realizar pruebas con los modelos arrojados al momento para saber exactamente cuál era el avance y la precisión actual con la que contábamos.



Esto llevó a la decisión de realizar una tercera fase de entreno la cual juntaría 15 épocas de entrenamiento y que nos servirá para saber si habría modificaciones en los datos. Esta fase arrojó una mejora exponencial en la métricas y se encontraron muchas mejoras realizando pruebas.

```
PS C:\Users\alejandros> & C:/Users/alejandros/AppData/Local/Programs/Python/Python313/python.exe c:/Users/alejandros/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 194.7±153.5 MB/s, size: 19.4 KB)
val: Scanning C:\Users\alejandros\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95)
all	4661	4661	0.889	0.883	0.951	0.929
organico	478	478	0.929	0.878	0.964	0.931
reciclable	2105	2105	0.884	0.945	0.972	0.956
otros	1340	1340	0.905	0.862	0.957	0.943
plastico	738	738	0.837	0.846	0.914	0.888

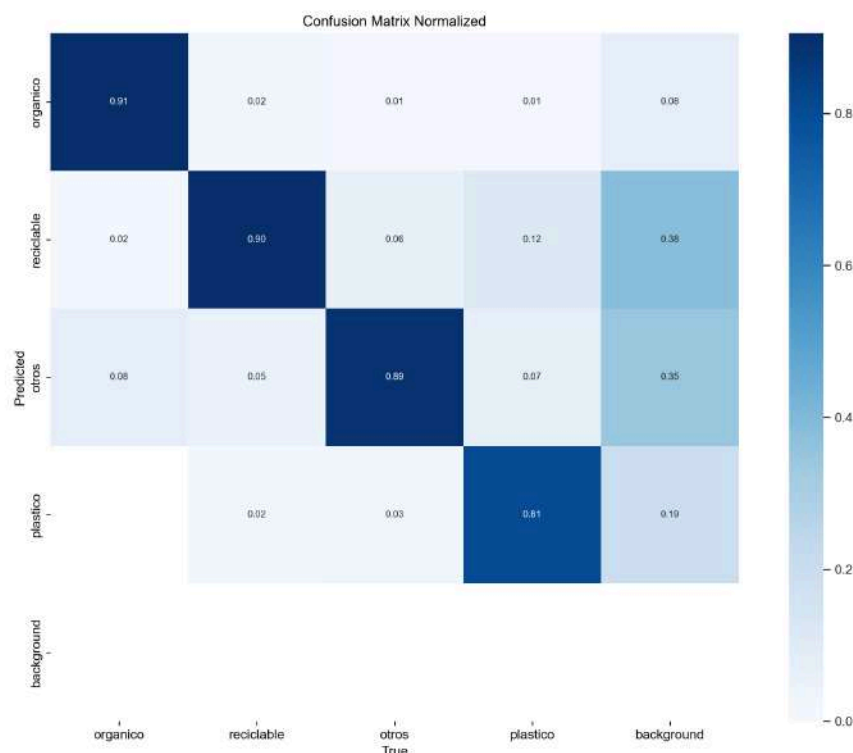
```
Speed: 0.7ms preprocess, 91.8ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val4
Precisión media (mp): 0.8887468359511743
Recall medio (mr): 0.8828433347672093
mAP50: 0.951488346102469
mAP50-95: 0.9294845371179582
Precisión por clase: [ 0.92916 0.88382 0.9054 0.83661]
Recall por clase: [ 0.87808 0.94489 0.86194 0.84646]
F1 por clase: [ 0.9029 0.91334 0.88313 0.8415]
mAP50 por clase: [ 0.96351 0.9719 0.95651 0.91402]
mAP50-95 por clase: [ 0.93116 0.956 0.94293 0.88784]
```

Sin embargo, con la intención de encontrar la mejor versión del modelo e intentando averiguar el por que del bajo rendimiento en una de las clases de detección, comenzó una nueva fase de investigación sobre posibles fallos más aparte de realizar un nuevo entrenamiento que contaría con 2 épocas y en el cual modificaremos nuestros datos para buscar un posible mejora.

Este entrenamiento de prueba resultó con peores en las métricas no afectadas anteriormente y nos sirvió como información para saber qué es lo que pasaba en nuestro modelo.

Debido a esto se llegó a la conclusión de que el modelo anterior, constituido por 15 épocas de entreno y que llegó a presentar las mejores cifras de rendimiento sería la opción final elegida.

Esta fase, me permitió entender la inmensa cantidad de gasto computacional que se genera para proyectos de este tipo y la importancia de un buen equipo para la realización de este tipo de tareas, aparte de aprender a analizar métricas de modelos machine learning y saber qué hacer con estas una vez recibidas. Otro punto importante fue el conocer el funcionamiento de un entrenamiento, ser consciente del tiempo que es necesario y lo fundamental de un dataset en forma para este proceso, aparte de aprender a modificar la información para poder solucionar fallos.



- Resultados del modelo final y pruebas.

Con el modelo finalizado y elegido se preparó una fase extensa de pruebas en la cual se analizó la precisión del modelo elegido más allá de las métricas obtenidas buscando realizar pruebas más realistas con imágenes que pudieran imitar el ambiente en el que se plantea ser utilizado y así conseguir una fiabilidad alta con el modelo. Esto fue hecho mediante la herramienta de Google Colab en la cual se pudo ejecutar código que pudiera usar el modelo para realizar pruebas mediante diferentes librerías.

```
[ ] |pip install ultralytics

[ ] | from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
from google.colab import files
from PIL import Image
import numpy as np

[ ] | uploaded = files.upload()

[ ] | model = YOLO('best1.pt')

[ ] | uploaded = files.upload()
Elegir archivos Sin archivos seleccionados Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

[ ] | image_path = '15.jpg'
results = model(image_path)
results[0].show()

[ ] | img = Image.open(image_path)
plt.imshow(np.array(img))
plt.axis('off')
plt.show()

[ ] | for result in results:
    for box in result.boxes:
        cls = model.names[int(box.cls)]
        conf = box.conf.item()
        print(f'Detectado: {cls} con confianza: {conf:.2f}')
Detectado: organico con confianza: 0.54

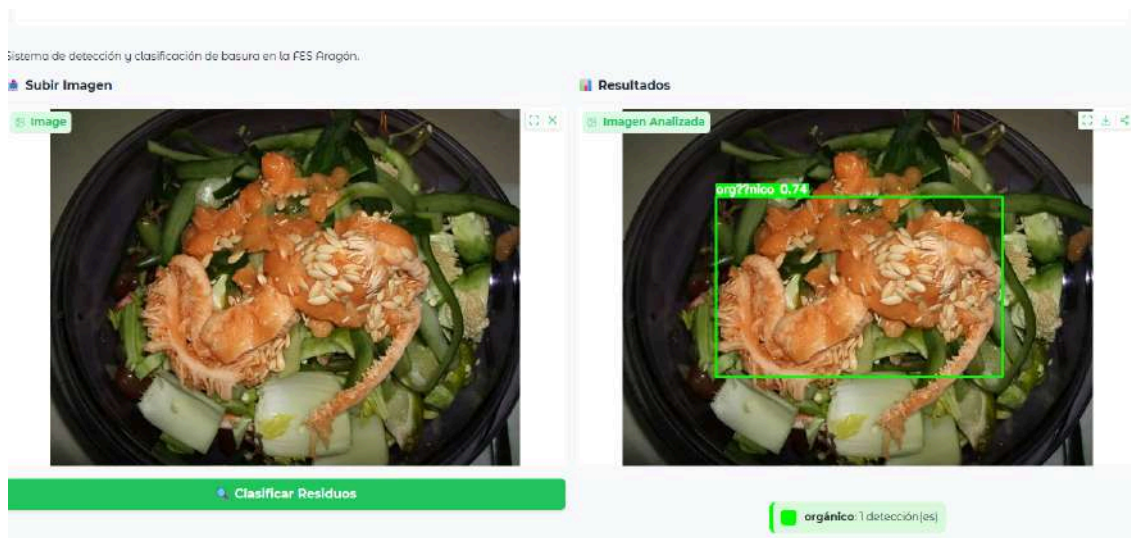
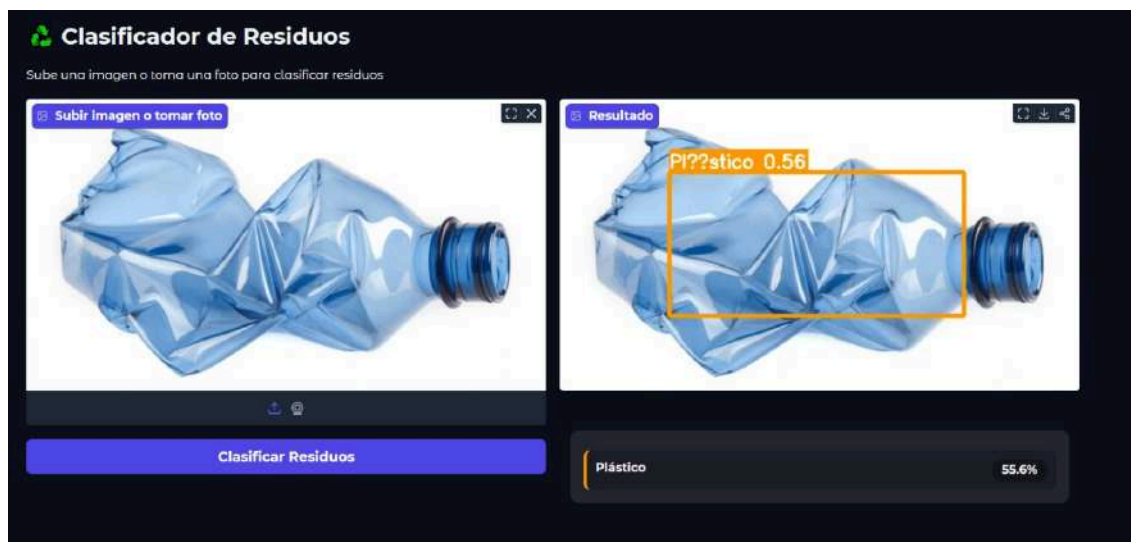
[ ] | from google.colab import files
files.download('/content/best.pt')
```

Los resultados exitosos en la precisión del modelo me permitieron dar luz verde a la siguiente fase del proyecto y aprender técnicas de implementación de modelos machine learning aparte de experiencia en el uso de herramientas como Colab.

- Presentación de modelo final para implementación en siguientes fases del proyecto y retroalimentación y apoyo.

Con todo terminado, me dispuse a servir como apoyo para la siguiente fase del proyecto, brindando retroalimentación acerca del funcionamiento del modelo y dataset, continuando realizando pruebas y aportando ideas en la implementación del modelo

Esta fase aunque no lo parezca fue importante, pues cerró el proyecto y me ayudó a comprender lo importante de la comunicación en el equipo dentro de un proyecto así, aparte de comprender que aun con todo terminado siempre hay más pruebas por hacer y más mejoras o errores que seguirán surgiendo en el camino.



Las actividades realizadas en este proyecto han sido importantes para poder evolucionar mis habilidades como estudiantes de la carrera de Ingeniería en Computación por parte de la FES Aragón, esta experiencia me ha brindando nuevas herramientas en el campo de la Inteligencia Artificial, contando ahora con una visión propia del funcionamiento de estas herramientas, obteniendo experiencia en su manejo y conocimiento de cómo funciona el flujo de trabajo en un proyecto de este calibre.



Responsable:
M. en C. Arturo Ocampo Álvarez
Prof. T.C IEE, FES Aragón UNAM
CIMA CUBÍCULO 118



Dr. José Paulo Mejía Mota
Secretario académico del programa de
Investigación FES Aragón UNAM



Alvarado Castillo Alejandro
No. Cta. 319110412
E-mail: alejandroalvarado412@aragon.unam.mx



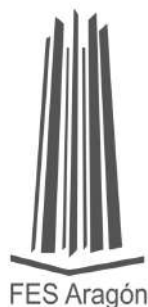
**UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO
FACULTAD DE ESTUDIOS SUPERIORES
ARAGÓN
CARRERA DE INGENIERÍA EN COMPUTACIÓN**

**CENTRO DE INVESTIGACIÓN
MULTIDISCIPLINARIA ARAGÓN
24 DE FEBRERO AL 11 DE SEPTIEMBRE DEL
2025**

**REPORTE FINAL DE SERVICIO SOCIAL
EcoAragón - DETECCIÓN DE BASURA CON IA
(RECOPIACIÓN DE DATOS, PREPARACIÓN DEL
DATASET, ENTRENAMIENTO)**

**PRESENTA:
ALVARADO CASTILLO ALEJANDRO**

**RESPONSABLE:
ING. OCAMPO ÁLVAREZ ARTURO**



ESTADO DE MÉXICO, SEPTIEMBRE 2025.

Índice

- Introducción	2
- Justificación del Proyecto	3
- Objetivos	4
- Descripción del trabajo	5
- Recolección y preparación de datos	6
- Progreso del entrenamiento	11
- Selección del modelo	18
- Problemas, soluciones y mejoras	20
- Conclusiones	21
- Bibliografía:	22

Introducción

El siguiente reporte expondrá la primera fase del proyecto EcoAragón – Detección de basura con IA, desarrollado dentro del Centro de Investigación Multidisciplinaria Aragón (CIMA). En este detallare las actividades realizadas durante esta etapa inicial, que constituye la base para el desarrollo y consolidación del proyecto en fases posteriores.

Esta etapa del proyecto tiene como objetivo desarrollar un modelo de machine learning capaz de reconocer distintos tipos de basura a partir de imágenes, facilitando así su clasificación automatizada. En esta primera etapa, me enfocaré en dos aspectos fundamentales: la recolección de información y la construcción de un dataset propio, seguido del diseño y entrenamiento del modelo. Estos pasos son esenciales, ya que la calidad de los datos influye directamente en el rendimiento y precisión del sistema de reconocimiento.

A lo largo de este reporte, se describe el proceso de recopilación de datos, la metodología utilizada para la creación y organización del dataset, así como el desarrollo y entrenamiento preliminar del modelo. Esta fase constituye la base para futuras mejoras y la implementación práctica de una solución orientada a la gestión inteligente de residuos.

Justificación

La correcta gestión de los residuos dentro de instituciones educativas, como lo es nuestra institución, es fundamental para mantener espacios limpios, seguros y sostenibles. Sin embargo, muchas veces la clasificación y el control de la basura se realizan de forma manual, lo que genera ineficiencias y errores que dificultan los procesos de reciclaje y el aprovechamiento adecuado de los desechos. Este problema no solo afecta el orden y la imagen de la institución, sino que también representa un reto ambiental y logístico que puede derivar en mayores costos de gestión y una menor conciencia ecológica en la comunidad universitaria.

Ante esta situación, el presente proyecto surge como una propuesta tecnológica que busca apoyar a la universidad en la optimización del control de sus residuos. En particular, esta primera etapa se enfoca en la construcción de la base sobre la cual se desarrollará todo el sistema: la recolección de información, la creación de un dataset propio y el entrenamiento inicial de un modelo de machine learning capaz de reconocer distintos tipos de basura a partir de imágenes.

La importancia de esta fase radica en que un modelo de clasificación sólo puede ser tan bueno como los datos con los que ha sido entrenado. Al recopilar información específica de nuestro contexto universitario, aseguramos que el sistema sea preciso y adaptado a las necesidades reales de la institución. De esta manera, se sientan los cimientos para un proyecto escalable que, en etapas posteriores, podrá integrarse en alguna plataforma o infraestructura capaz de contribuir a una gestión de residuos más eficiente, sostenible y alineada con los objetivos ecológicos de la comunidad educativa.

Objetivos

Objetivo General:

Desarrollar la primera etapa de un sistema inteligente para la clasificación automatizada de residuos en la universidad, mediante la recolección y organización de datos, la creación de un dataset propio y el entrenamiento inicial de un modelo de machine learning, con el fin de establecer una base sólida para las fases posteriores del proyecto.

Objetivos Específicos:

- Recolectar información visual de los distintos tipos de residuos generados en la universidad, asegurando que las imágenes sean representativas y de calidad para el proceso de entrenamiento.
- Diseñar y estructurar un dataset propio, categorizando correctamente las imágenes según el tipo de basura (orgánica, inorgánica, reciclable, etc.), garantizando que el modelo pueda aprender de manera precisa.
- Entrenar un modelo inicial para el reconocimiento básico de los diferentes tipos de residuos, evaluando su desempeño preliminar y detectando áreas de mejora.
- Documentar los procesos realizados durante esta fase, incluyendo la metodología de recolección de datos, la construcción del dataset y el entrenamiento del modelo, con el propósito de facilitar la continuidad del proyecto en etapas futuras.
- Establecer los fundamentos técnicos y metodológicos que permitan escalar el sistema, de forma que pueda implementarse posteriormente.

Estos objetivos buscan garantizar que el proyecto cuente con una base sólida, enfocada en datos de calidad y un modelo adaptable a las necesidades específicas de la universidad, asegurando así la viabilidad de su implementación a largo plazo.

Descripción del Trabajo

Recolección de información

Antes de comenzar con el trabajo complete una fase de investigación y estudio acerca del panorama general de la Inteligencia Artificial para usos parecidos en nuestro objetivo, después de analizar las herramientas disponibles confirme que la mejor herramienta disponible para nuestro proyecto es el modelo *YOLOv8*.

Hardware:

Tras planear los primeros requerimientos comencé a buscar cual sería la mejor opción en tanto a hardware para la realización de una tarea tan exigente como el desarrollo de un modelo de Machine Learning.

Considere diferentes opciones como por ejemplo el préstamo de hardware especializado o el uso de herramientas online como *Colab* de google, sin embargo, después de realizar algunas pruebas iniciales y tomando en cuenta que para el uso de dispositivo online se necesitaría de versiones premium más el hecho de que afortunadamente contaba con equipo de potencia relativamente alta decidí recurrir al uso de un dispositivo local con las siguientes características:

- Tarjeta de video Radeon RX 6600 8GB GDDR6
- Procesador AMD Ryzen 5 5500 / 6 Core / 12 Thread / 3.6GHz
- Memoria RAM ADATA XPG GAMMIX D35 DDR4 16GB 3200MHZ (x2)

Este proceso hizo que lograra darme cuenta de la exigencia en coste computacional que conlleva realizar un proyecto basado en Inteligencia Artificial de este calibre.

Recolección y preparación de datos

Después de la investigación que realice sabía que para el entrenamiento de un modelo especializado y con las características específicas del ambiente en el que será funcional me era necesaria la ayuda de un dataset de imágenes que contará con diferentes características específicas del proyecto.

- Cumplimiento de categorías según el espacio

Plantee que este dataset contaría con cuatro diferentes categorías o “labels” proporcionadas por los estándares de reciclaje de la institución: Plásticos, Orgánicos, Reciclable y Otros.

Cada una de estas etiquetas podría contar con diferentes tipos de basura, en este caso por orden:

1. Orgánica:
 - a. Semillas, Restos de comida, Cáscaras, Palitos de Madera(usados), Bolsas de té, Residuos de Café y Huesos.
2. Reciclable:
 - a. Vidrio, Papel(Folders, Hojas usadas o dobladas), Revistas, Carton, Cajas (carton o tipo tetrapack), Latas(refresco, conservas, enlatados, etc), Papel Aluminio(restos o envases), Metal y Bolsas de papel.
3. Plástico
 - a. Botellas de diferentes tipos y extensiones de este tipo de plásticos.
4. Otros:
 - a. Cubrebocas, Cepillos, Pañales, Bolsas de plástico, Envolturas de plástico laminadas o metalizadas, “wrappers”, Popotes, Grapas, Cigarros, Servilletas usadas, Cartón con grasa, Charolas, Platos, Cubiertos, Vasos, y Variantes de Unicef/Plástico.

Todas las decisiones acerca de la forma en la que construiría el dataset las hice tomando en cuenta como se supone que funciona el tratamiento de los residuos dentro de la institución y me hicieron darme cuenta la importancia de estudiar correctamente la problemática y entenderla para así realizar un trabajo más eficiente.

- Tamaño

Para lograr un entrenamiento exitoso, con porcentajes ideales, se requiere de una cantidad de datos enormes y bien preparados. Cada categoría contó con una cantidad considerable de imágenes tanto para su etapa “train” como “val”.

Orgánica: 2,441 imágenes.

Reciclable: 10,542 imágenes.

Plástico: 3,371 imágenes.

Otros: 6,503 imágenes.

Dando un total de 22,857 imágenes en total dentro del dataset.

- Procesamiento y Formato

Al utilizar un modelo YOLOv8 para el entrenamiento el dataset debía de contar con un formato específico en su carpeta:

```
dataset/
├── images/
│   ├── train/
│   │   ├── img1.jpg
│   │   ├── img2.jpg
│   │   └── val/
│   │       ├── img3.jpg
│   │       └── img4.jpg
│   └── labels/
│       ├── train/
│       │   ├── img1.txt
│       │   ├── img2.txt
│       │   └── val/
│       │       ├── img3.txt
│       │       └── img4.txt
│   └── data.yaml
```

Este formato es requerido por YOLO para crear una detección de objetos. Dentro de la carpeta “images” se encuentran divididas todas las imágenes en un porcentaje 80-20 entre “train” y “val” respectivamente y dentro de la carpeta “labels” los respectivos archivos .txt de cada imagen. El archivo final llamado “data.yaml” sirve como descriptor del dataset.

- Herramientas De Recolección

Para la tarea de recolección de tantos datos recurrí a diferentes herramientas creadas para este mismo propósito.

1. Datasets de plataforma

Para comenzar dispuse de plataformas como Kaggle, Hugging Face, RoboFlow y GitHub.

Tanto para recolección de datos como para información y ejemplos de funcionamiento estas plataformas fueron indispensables en la recolección.



2. Extension de Google: Fatkun Batch

Esta extensión, creada específicamente con este objetivo, permite descargar cúmulos de imágenes con solo ser cargadas por el navegador e incluso modificar sus nombres y extensiones fácilmente.



3. Pequeño dataset personal con fotos tomadas de los residuos más comunes en la facultad.

- Renombramiento y creación de labes con Scripts

Para automatizar un proceso lento y repetitivo como la creación de labels y el renombramiento de imágenes me dispuse a la creación de un script que automatiza este proceso para poder realizarlo en segundos.

```
import os
from PIL import Image
import shutil

input_folder = r"C:\Users\alejandro\Downloads\xd"
output_folder = r"C:\Users\alejandro\Downloads\xd\salida"

os.makedirs(output_folder, exist_ok=True)

valid_exts = ('.jpg', '.jpeg', '.png')

prefix = "plastico_00007"
count = 0

for filename in sorted(os.listdir(input_folder)):
    if filename.lower().endswith(valid_exts):
        old_path = os.path.join(input_folder, filename)

        new_name = f"{prefix}{count:02d}.jpg"
        new_path = os.path.join(input_folder, new_name)

        if not filename.lower().endswith(".jpg"):
            img = Image.open(old_path).convert("RGB")
            img.save(new_path, "JPEG")
            os.remove(old_path)
        else:
            os.rename(old_path, new_path)

        label_name = new_name.replace(".jpg", ".txt")
        label_path = os.path.join(output_folder, label_name)

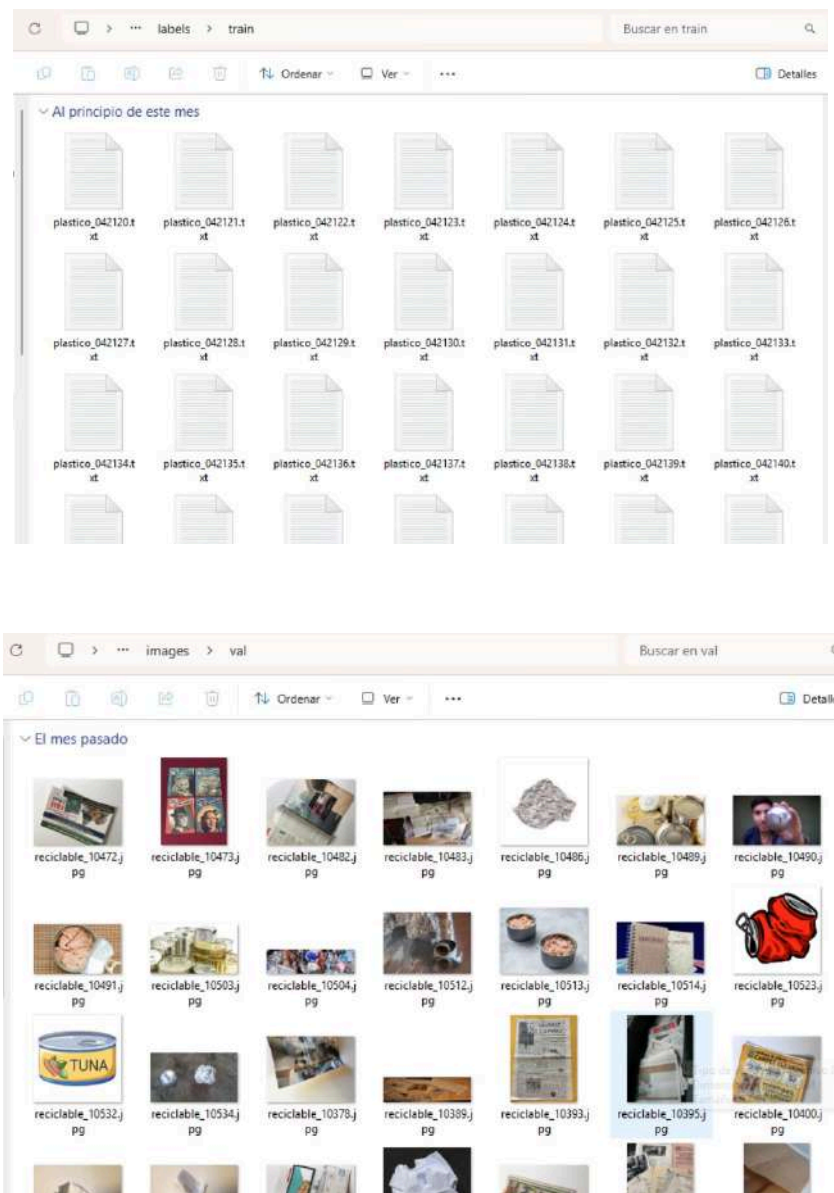
        with open(label_path, "w") as f:
            f.write("3 0.5 0.5 0.5 0.5\n")

        count += 1

print(f"Renombradas {count} imágenes y generadas sus etiquetas en '{output_folder}'")
```

Todo este proceso resultó increíblemente laborioso y me dio una pequeña introducción a lo que es trabajar con cantidades enormes de datos y lo cuidadoso que hay que ser con ellos.

Ejemplo del dataset de imágenes y etiquetas:



El tiempo aproximado de este proceso llega a una semana completa de trabajo durante todo el día, ya que al contar con cantidad de datos tan inmensa se requirió de mucho tiempo para completar los datos en forma y completos aparte de todos los tiempos de investigación, elección y planeación.

Progreso del Entrenamiento

El entrenamiento de un modelo es un proceso largo y con alto coste informático, por eso se requiere de computadores con gran capacidad de procesamiento.

- 1er entrenamiento

El primer proceso de entrenamiento del modelo fue planeado como una prueba, me decidí por un recorrido de 5 épocas (que servirán como checkpoints en caso de algún error y abierto al aumento de épocas futuras), en lotes de 32 imágenes, y un redimensionado de imágenes de 640px.

```
from ultralytics import YOLO
import os
import shutil

model = YOLO('yolov8s.pt')

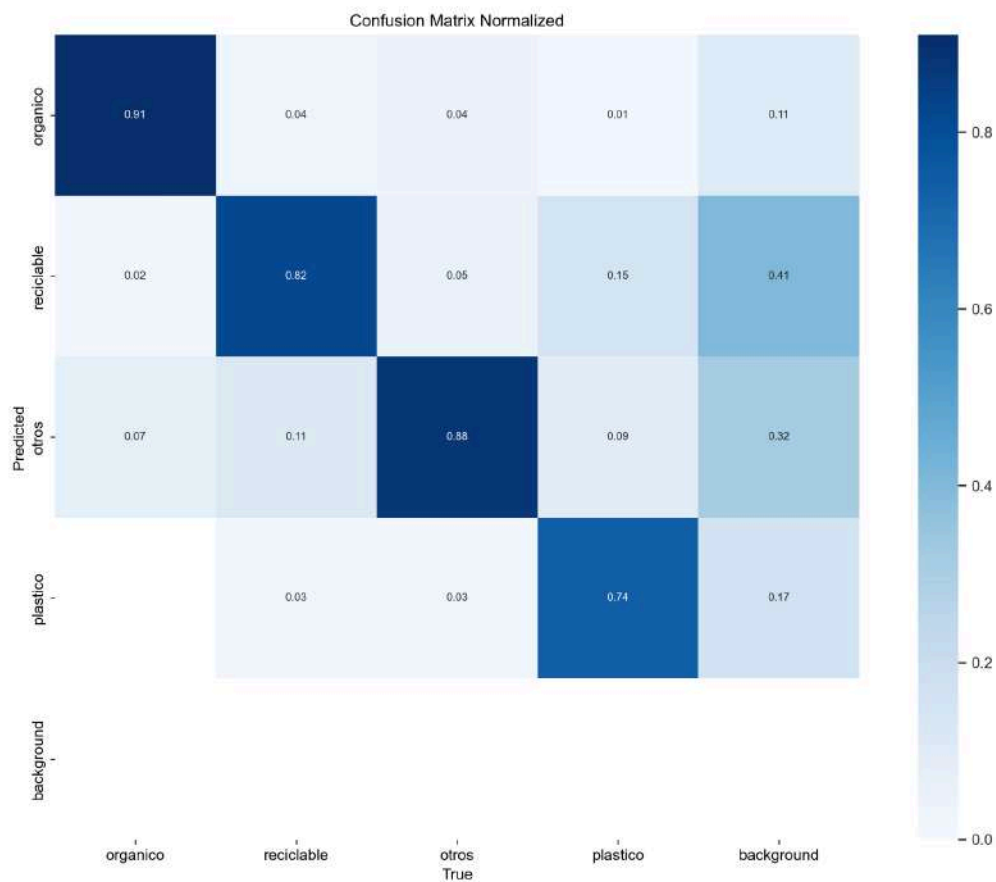
model.train(
    data= r'C:\Users\alejandro\Downloads\dataset_renamed_yolo\data.yaml',
    epochs=5,
    imgsz=640,
    batch=32
)
```

Este entrenamiento fue pausado con la herramienta “.last” de ultralytics, la cual me permitio guardar los parámetros de los pesos dados hasta el momento de la pausa y después continuar.

Para medir las mejores métricas decidí centrarme en *precision*, *recall*, *mAP50* y la *CMN* integrada en el archivo del entrenamiento.

mAP50 = mean average precision -> Esta es la métrica que más nos interesa porque refleja de forma sencilla y directa que tan bien detecta y clasifica el modelo.

Los resultados finales de este entrenamiento de 5 épocas fueron los siguientes.



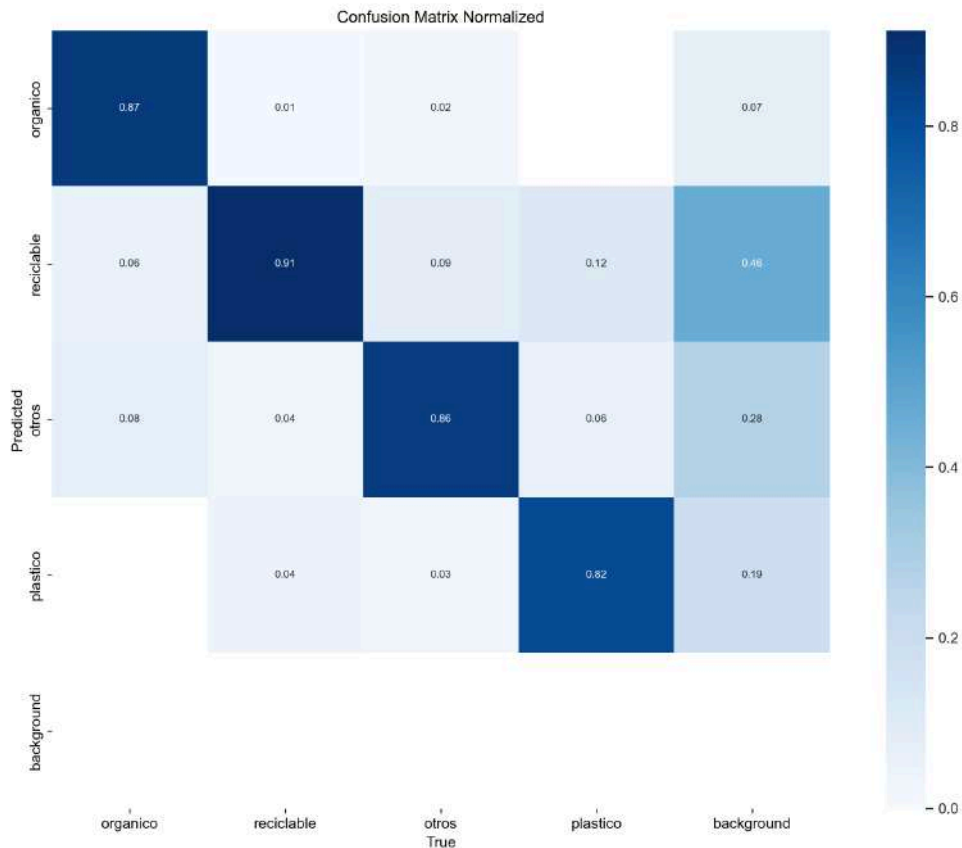
```
PS C:\Users\alejandro> & C:/Users/alejandro/AppData/Local/Programs/Python/Python313/python.exe c:/Users/alejandro/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 27.5±19.7 MB/s, size: 13.9 KB)
val: Scanning C:\Users\alejandro\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]
Class  Images  Instances  Box(P  R  mAP50  mAP50-95): 100%|██████████| 292/292 [07:32<00:00, 1.55s/it]
  all      4661      4661    0.869  0.859  0.941  0.929
  organico  478        478    0.8    0.916  0.946  0.941
  reciclable 2105      2105    0.935  0.844  0.959  0.946
  otros    1340      1340    0.83   0.89   0.941  0.93
  plastico  738        738    0.911  0.787  0.917  0.899
Speed: 0.7ms preprocess, 91.1ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val
Precisión media (mp): 0.8689179026599186
Recall medio (mr): 0.8591690833845309
mAP50: 0.9405667882189012
mAP50-95: 0.9290407483464488
Precisión por clase: [ 0.8    0.935  0.83009  0.91059]
Recall por clase: [ 0.91632  0.84418  0.88958  0.7866]
F1 por clase: [ 0.85422  0.88727  0.85881  0.84406]
mAP50 por clase: [ 0.94599  0.95892  0.94079  0.91656]
mAP50-95 por clase: [ 0.9415  0.94562  0.92993  0.89912]
```

Estos datos nos arrojan que el promedio de efectividad global del modelo es de un 80-85%, lo cual es bastante bueno, pero noté rápidamente que la clase de plásticos tenía un porcentaje débil a comparación de las demás, por lo que decidí seguir entrenando el modelo.

- 2do entrenamiento

Para continuar el proceso continúe sumando etapas de entrenamiento de 5 en 5 épocas.

Esto dio como resultado que se encontró una mejora en los porcentajes de cada una de las métricas.



```
PS C:\Users\alejandros> & C:\Users\alejandros\AppData\Local\Programs\Python\Python313\python.exe c:\Users\alejandros\OneDrive\Documents\entrenamiento_modelo\validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 78.0±42.0 MB/s, size: 12.0 KB)
val: Scanning C:\Users\alejandros\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]
Class  Images  Instances  Box(P  R      mAP50  mAP50-95): 100%|██████████| 292/292 [07:34<00:00, 1.56s/it]
all      4661      4661      0.891  0.885  0.95    0.911
organico 478       478       0.89   0.91   0.95    0.9
reciclable 2105     2105     0.911  0.925  0.965   0.922
otros    1340     1340     0.868  0.886  0.954   0.927
plastico 738       738       0.894  0.819  0.933   0.896
Speed: 0.7ms preprocess, 92.0ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val3
Precisión media (mp): 0.8905213205533233
Recall medio (mr): 0.885150620682896
mAP50: 0.9504117167499988
mAP50-95: 0.9113464106085303
Precisión por clase: [ 0.88996 0.91097 0.86756 0.8936]
Recall por clase: [ 0.91004 0.92542 0.88582 0.81932]
F1 por clase: [ 0.89989 0.91814 0.8766 0.85485]
mAP50 por clase: [ 0.94975 0.96451 0.9543 0.93308]
mAP50-95 por clase: [ 0.90024 0.92175 0.92721 0.89619]
```

- Pruebas intermedias

Para asegurar el correcto funcionamiento del modelo y gracias a la información encontrada en mi investigación decidí realizar pruebas del modelo mediante la plataforma de *Google Colab* en la cual utilice código python y diferentes librerías para realizar pruebas mediante imágenes.

```
[ ] | pip install ultralytics

[ ] | from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
from google.colab import files
from PIL import Image
import numpy as np

[ ] | uploaded = files.upload()

[ ] | model = YOLO('best1.pt')

[ ] | uploaded = files.upload()
Elegir archivos Sin archivos seleccionados Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

[ ] | image_path = '15.jpg'
results = model(image_path)
results[0].show()

[ ] | img = Image.open(image_path)
plt.imshow(np.array(img))
plt.axis('off')
plt.show()

[ ] | for result in results:
    for box in result.boxes:
        cls = model.names[int(box.cls)]
        conf = box.conf.item()
        print(f'Detectado: {cls} con confianza: {conf:.2f}')

Detectado: organico con confianza: 0.54

[ ] | from google.colab import files
files.download('/content/best.pt')
```

Aprendí bastante acerca del manejo de un modelo al menos mediante consola realizando estas pruebas y aunque dado estos entrenamientos seguí encontrando mejoras también me percaté que había errores en una clase que estaba fallando más que las demás después de llegar a las 10 épocas, donde encontramos errores que no deberían aparecer.

Fallos usuales



Figuras que no
están bien
definidas o podrían
confundirse
fácilmente

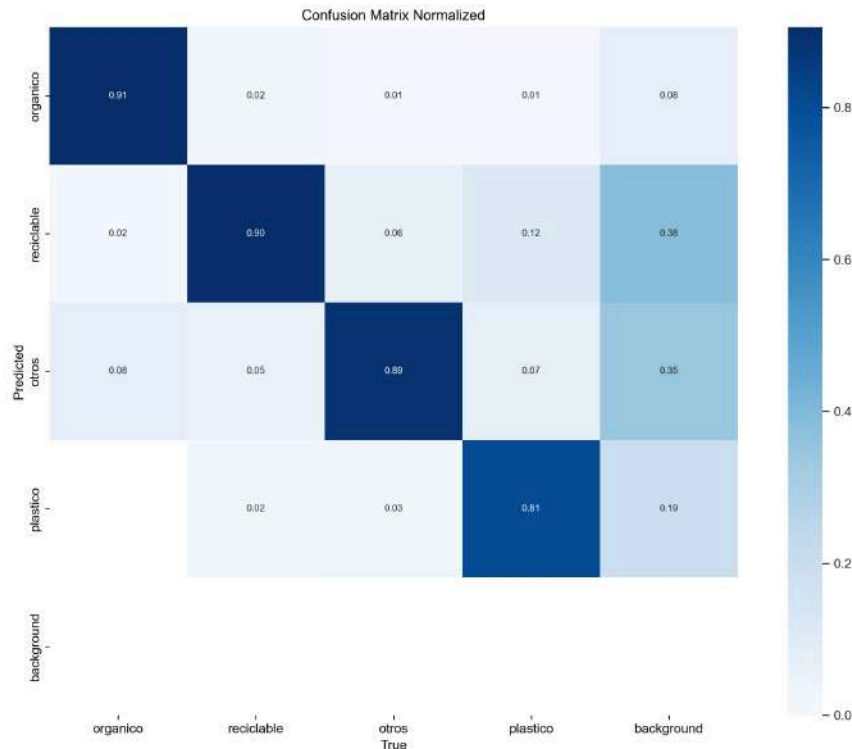


Muchos objetos en
la imagen

Debido a esto decidí que se realizaría un proceso de 5 épocas para evaluar el crecimiento de plásticos y conocer más acerca de los errores encontrados hasta el momento, si no se veía mejora recurriría a una modificación a los datos para buscar una mejora en las clases afectadas.

- 3er entrenamiento

Ahora, para el tercer entrenamiento, que en total sumaría 15 épocas, se encontraron las mejores métricas de evaluación que logramos para algunas clases, sin embargo “plástico” seguía siendo la clase más débil aunque realizando pruebas era bastante fiable.

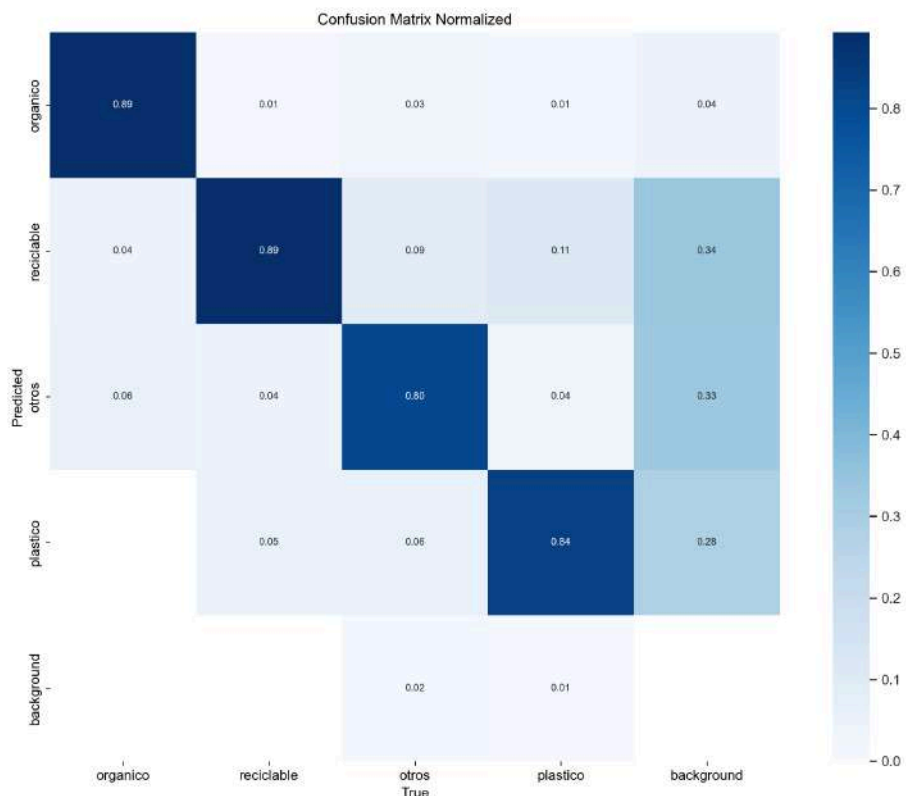


```
PS C:\Users\alejandro> & C:/Users/alejandro/AppData/Local/Programs/Python/Python313/python.exe c:/Users/alejandro/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 194.7±153.5 MB/s, size: 19.4 KB)
val: Scanning C:\Users\alejandro\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]
Class      Images  Instances  Box(P)   R      mAP50  mAP50-95: 100%|██████████| 292/292 [07:33<00:00, 1.55s/it]
all         4661      4661      0.889    0.883    0.951    0.929
organico    478       478      0.929    0.878    0.964    0.931
reciclable  2105     2105      0.884    0.945    0.972    0.956
otros       1340     1340      0.905    0.862    0.957    0.943
plastico    738       738      0.837    0.846    0.914    0.888
Speed: 0.7ms preprocess, 91.8ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val4
Precisión media (mp): 0.8887468359511743
Recall medio (mr): 0.8828433347672093
mAP50: 0.951488346102469
mAP50-95: 0.9294845371179582
Precisión por clase: [ 0.92916 0.88382 0.9054 0.83661]
Recall por clase: [ 0.87808 0.94489 0.86194 0.84646]
F1 por clase: [ 0.9029 0.91334 0.88313 0.8415]
mAP50 por clase: [ 0.96351 0.9719 0.95651 0.91402]
mAP50-95 por clase: [ 0.93116 0.956 0.94293 0.88784]
```

- 4to entrenamiento

Finalmente, para investigar un poco más acerca del funcionamiento del dataset decidí realizar un ajuste a este mismo y agregar más valores tanto a “/images” y “/val” para la clase “plástico” con la esperanza de comprobar si la evaluación no resultaba efectiva o hacia falta una mayor cantidad de datos para esta clase.

En todo caso se esperaban mejores resultados y por parte de “plástico” los hubo pero este último entrenamiento empeoró nuestras demás clases.



```
PS C:\Users\alejandro> & C:/Users/alejandro/AppData/Local/Programs/Python/Python313/python.exe c:/Users/alejandro/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 169.5±157.7 MB/s, size: 21.4 KB)
val: Scanning C:\Users\alejandro\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100% | 4661/4661 [00:00<?, ?it/s]

```

Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	4661	4661	0.862	0.864	0.924	0.876
organico	478	478	0.9	0.882	0.928	0.902
reciclable	2105	2105	0.887	0.907	0.952	0.898
otros	1340	1340	0.892	0.811	0.918	0.869
plastico	738	738	0.769	0.854	0.897	0.834

```
Speed: 0.7ms preprocess, 92.9ms inference, 0.0ms loss, 0.3ms postprocess per image
Results saved to runs\detect\val5
Precisión media (mp): 0.8619211535814406
Recall medio (mr): 0.8636111038345791
mAP50: 0.9239645048427011
mAP50-95: 0.8756350150062426
Precisión por clase: [ 0.89972 0.88688 0.89201 0.76907]
Recall por clase: [ 0.88223 0.90736 0.81119 0.85366]
F1 por clase: [ 0.89089 0.897 0.84968 0.80916]
mAP50 por clase: [ 0.92848 0.95167 0.91842 0.89728]
mAP50-95 por clase: [ 0.90182 0.89757 0.8687 0.83445]
```

Selección de modelo

A partir de este 4to entrenamiento (17 épocas) note que estaba experimentando algo que conocí en mi investigación como Overfitting, que es cuando un modelo aprende demasiado bien los datos de entrenamiento, hasta el punto de memorizar sus patrones específicos y ruidos, en lugar de aprender reglas generales que sirvan para datos nuevos.

Después de buscar información referente y realizar pruebas, note que hasta el entrenamiento de 15 épocas obtuve el mejor modelo que entrene, pero la clase de plásticos nos daba un porcentaje bajo en el resumen del modelo aunque una alta eficacia en las pruebas, por lo que concluí que el error estaba en la carpeta “val” en la cual el modelo hace las pruebas, por lo que decidí tomar el modelo con 15 épocas de entrenamiento para el uso de nuestro proyecto.

Este es el modelo final con una eficacia global del 95 %.

```
PS C:\Users\alejandro> & C:/Users/alejandro/AppData/Local/Programs/Python/Python313/python.exe c:/Users/alejandro/OneDrive/Documents/entrenamiento_modelo/validacion_m.py
Ultralytics 8.3.111 Python-3.13.3 torch-2.6.0+cpu CPU (AMD Ryzen 5 5500)
Model summary (fused): 72 layers, 11,127,132 parameters, 0 gradients, 28.4 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 194.7±153.5 MB/s, size: 19.4 KB)
val: Scanning C:\Users\alejandro\Downloads\dataset_renamed_yolo\labels\val.cache... 4661 images, 0 backgrounds, 0 corrupt: 100%|██████████| 4661/4661 [00:00<?, ?it/s]

```

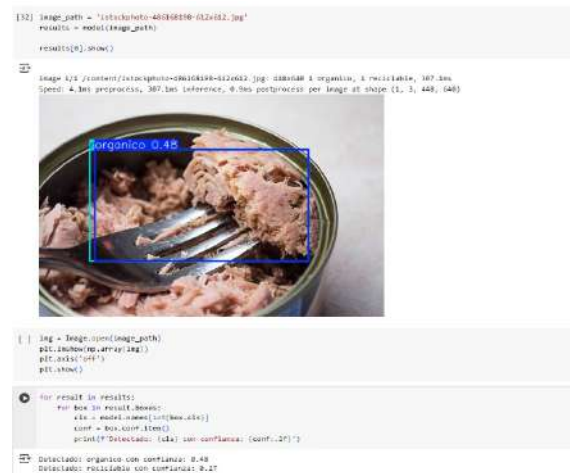
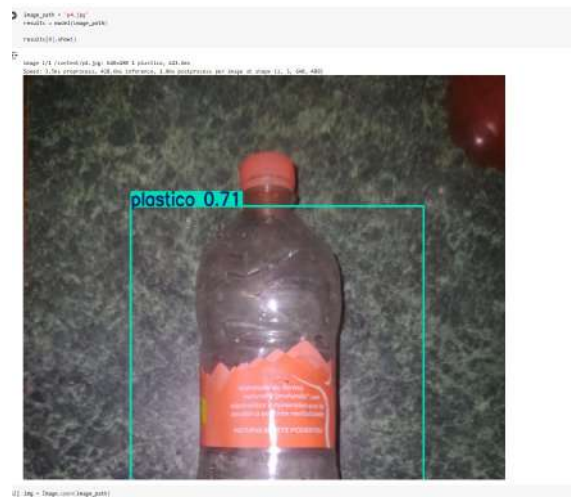
Class	Images	Instances	Box(P)	R	mAP50	mAP50-95
all	4661	4661	0.889	0.883	0.951	0.929
organico	478	478	0.929	0.878	0.964	0.931
reciclable	2105	2105	0.884	0.945	0.972	0.956
otros	1340	1340	0.905	0.862	0.957	0.943
plastico	738	738	0.837	0.846	0.914	0.888

```
Speed: 0.7ms preprocess, 91.8ms inference, 0.0ms loss, 0.2ms postprocess per image
Results saved to runs\detect\val4
Precisión media (mp): 0.8887468359511743
Recall medio (mr): 0.8828433347672093
mAP50: 0.951488346102469
mAP50-95: 0.9294845371179582
Precisión por clase: [ 0.92916 0.88382 0.9054 0.83661]
Recall por clase: [ 0.87808 0.94489 0.86194 0.84646]
F1 por clase: [ 0.9029 0.91334 0.88313 0.8415]
mAP50 por clase: [ 0.96351 0.9719 0.95651 0.91402]
mAP50-95 por clase: [ 0.93116 0.956 0.94293 0.88784]
```

Cabe recalcar que este proceso duró alrededor de una semana de trabajo continuo durante días completos (no de forma seguida) ya que las etapas de entrenamiento tomaban cerca de 15 horas para completar 5 épocas en el cual me mantenía realizando pruebas y recopilando más información, aparte de obviamente vigilar que el proceso siguiera de forma correcta.

- Pruebas finales

Aun así, para no apresurar el proyecto y encontrar futuros fallos, ya con el modelo elegido, me dispuse a la tarea de realizar una cantidad alta de pruebas, tanto con imágenes que pudiera recolectar de la internet, como de imágenes tomadas por cuenta propia y en la cual pudiera reflejar un uso real y el cual me diera un panorama de la asertividad del modelo más allá de las métricas.



Después de confirmar que contábamos con porcentaje de acierto que satisficiera los requerimientos del proyecto proseguí a comunicarme con los demás integrantes del equipo para que pudieran continuar con la siguiente fase y brindar apoyo en dudas sobre el funcionamiento del proyecto.

Problemas, soluciones y mejoras

Se encontraron dos problemas principales.

- Tiempo de entrenamiento:
Estime que el tiempo destinado a los entrenamientos a pesar de ser dividido en fases es demasiado largo llegando a tardar 12-14 horas por fase lo cual no permite realizar muchas pruebas en los entrenamientos a menos que sean pausados.
- Datos:
Aunque contamos con una cantidad mayor a las 20,000 imágenes, creo que la calidad de los datos de entrenamiento privó el proyecto de un modelo aún mejor del que logré.

Principales puntos de solución.

- Augmentation (técnica para replicar imágenes con diferentes ángulos e iluminación)
- Pruebas de campo
- Datos Realísticos

Mejoras futuras:

- Este proyecto es factible para futuras expansiones, como la integración con cámaras en tiempo real, sistemas físicos de clasificación automatizada en cada zona donde se ubican los contenedores de basura o implementación en apps o páginas web.
- Aumento del dataset para obtener mejores resultados.
- Creación de un entorno que permita a los mismos usuarios subir imágenes para robustecer el dataset.

Conclusiones

La finalización de esta primera fase del proyecto representa un avance significativo hacia la solución del problema de gestión de residuos en la universidad. A través de la recolección y organización de datos, la creación de un dataset propio y el entrenamiento de un modelo de machine learning, se logró desarrollar un sistema inicial capaz de clasificar distintos tipos de basura con un alto nivel de precisión.

La capacidad de crear un dataset tan completo y extenso que luego fue implementado exitosamente para un entrenamiento brindó la confianza necesaria para realizar tareas tan extensas en tiempo y de tanto coste.

El hecho de que el modelo alcanzará un buen rango de acierto demuestra la efectividad del proceso seguido y valida la importancia de haber invertido tiempo y esfuerzo en la calidad de los datos. Esta precisión inicial no solo confirma la viabilidad técnica del proyecto, sino que también sienta las bases para su escalabilidad y futura implementación en entornos reales dentro de la universidad.

Además, los resultados obtenidos brindan información valiosa para la siguiente etapa, donde se buscará una implementación de este.

En conclusión, esta primera fase ha sido exitosa al cumplir su objetivo principal: establecer un modelo confiable y adaptado al contexto universitario, que permitirá, en el futuro, mejorar la eficiencia del control de basura, reducir los errores en la clasificación y fomentar prácticas más sostenibles dentro de la comunidad académica.

Bibliografía

- Parul Pandey. (201) *Fatkun Batch Download Image Demo* [Video]. YouTube. Recuperado de <https://www.youtube.com/watch?v=O0gMgveGWWhs> YouTube
- Deep Lizard. (2025). *Depth First Search in Python – algorithm explained & coding demo* [Video]. YouTube. Recuperado de https://youtu.be/RRB7tS8VsDU?si=XXjqukeNynN_MGzO YouTube
- Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8* (versión 8.0.0) [Software]. Ultralytics. Recuperado de <https://docs.ultralytics.com/es/models/yolov8/> docs.ultralytics.com
- TensorFlow. (2024). *TensorFlow Documentation*. Google. Recuperado de: <https://www.tensorflow.org>
- Rafa Gonzalez Gouveia. (aprox. 4.3 años atrás). *¿Dónde Encontrar DATOS para Proyectos Data Science? ¿Qué es Kaggle?* [Video]. YouTube. Recuperado de <https://www.youtube.com/watch?v=NhHTWGlglRI> YouTube
- Canal o autor no especificado. (aprox. 5.6 años atrás). *Tutorial: GOOGLE COLAB e Inteligencia Artificial* [Video]. YouTube. Recuperado de <https://www.youtube.com/watch?v=0mOV4plF2Xo> YouTube
- ¿Qué es el sobreajuste? - Explicación del sobreajuste en machine learning - AWS. (n.d.). Amazon Web Services, Inc. <https://aws.amazon.com/es/what-is/overfitting/>
- IBM. (s. f.). *What is a dataset?* IBM Think. Recuperado de <https://www.ibm.com/think/topics/dataset>

- GeeksforGeeks. (s. f.). *What is a dataset: Types, features, and examples.*

Recuperado de <https://www.geeksforgeeks.org/data-science/what-is-dataset/>

- Kaggle. (s. f.). *Making datasets.* Kaggle Discussions. Recuperado de

<https://www.kaggle.com/discussions/general/44119>

- Hugging Face. (s. f.). *Create a dataset.* Hugging Face Datasets documentation. Recuperado de

https://huggingface.co/docs/datasets/create_dataset

- Analytics Vidhya. (s. f.). *Data representation in neural networks – tensor.*

Recuperado de

<https://www.analyticsvidhya.com/blog/2022/07/data-representation-in-neural-networks-tensor/>